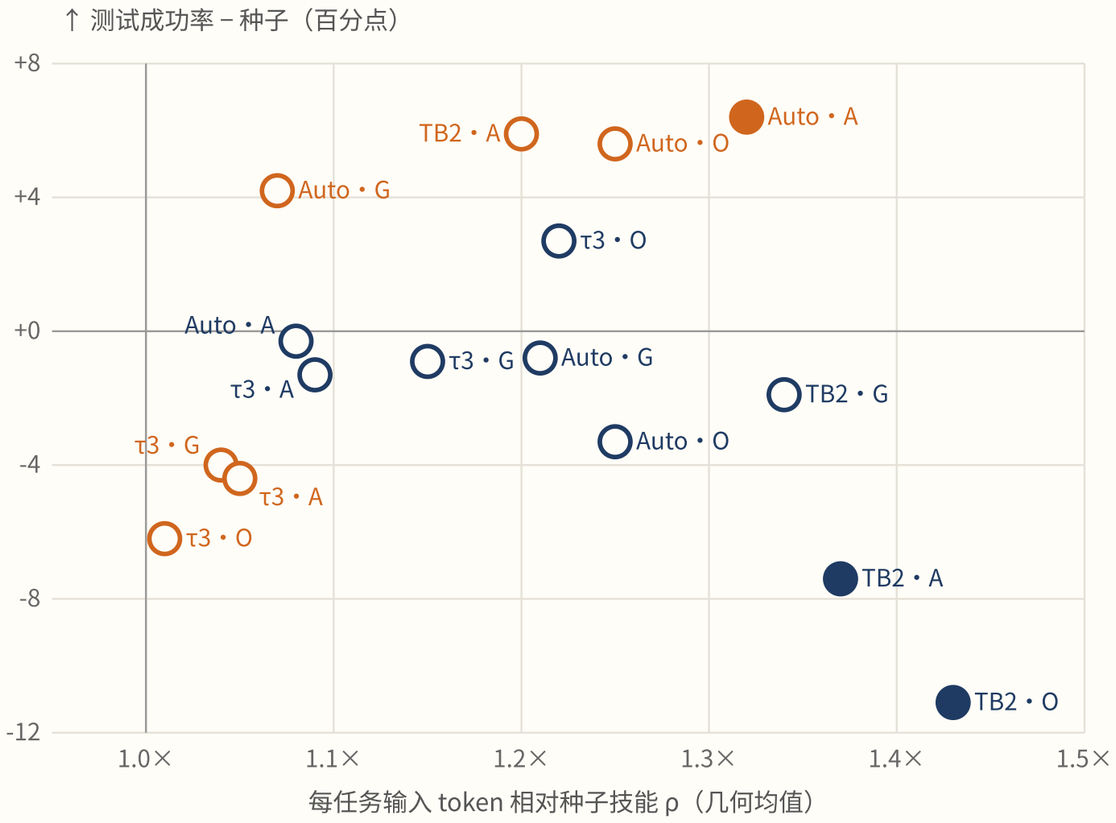
arXiv:2610.37201v1 · cs.AI · 2026-10-27 · Zhou, Peng, Huang, Zhang, Sun, Wang, Lu（上海 AI Lab 等）

技能优化器报出的增益是选出来的还是学出来的？多出来的费用又记在哪一行？

GEPA、SkillOpt、SkillAdam 改写放进上下文的“技能”文本，在 ⩽ 30 个开发任务（TB2 只有 9 个）上分数更高就保留。长时程任务成功率噪声大、上下文每回合重发——BiPA 不改优化器，只用日志回答两件事：**开发增益里有多少是挑选本身造成的；多读的 token 来自哪种行为**。

小票 01 · 测试结果（avg@3） Table 1 · 附录 C.1–C.2

16 个学到的技能：成功率不动，读的更多



● DeepSeek-V4.1-Flash ● GPT-6-Luna · 标签：基准 · 优化器（G = GEPA，O = SkillOpt，A = SkillAdam）
实心：与种子的差距超过种子自身三次测试运行的极差（附录 C.2）

16 个技能合计 vs 种子（单侧 95% 上界）	+0.1 点（+1.6）
DeepSeek 9 个合计	−1.4 点（+0.6）
输入 token 比 ρ（16 个合计）	1.16×

Luna 在 TB2 上 GEPA、SkillOpt 都返回了种子，故 18 个运行只有 16 个技能。唯一**真涨**的是 Luna 的 AutoBench（+4.2 至 +6.4；只有 SkillAdam 的 +6.4 超过种子极差 5.8）。DeepSeek 9 个技能里 8 个的**每次**测试运行都比种子任何一次读得多。

小票 02 · 选择回放 §4.1 · Fig. 3 · 附录 C.3–C.4

把“候选”换成没改过的种子，照样中奖

按每次运行实际的候选数与接受规则，把每个候选都当作**原封不动的种子**重新打分 10 万次，看仅靠挑选能报出多大的开发增益 G_0。

DeepSeek · SkillOpt / GEPA 报告增益被回放达到的概率	0.56–0.94
DeepSeek · SkillAdam（TB2 / τ^3 / AutoBench）	0.25 / 0.87 / 0.88
Luna（多种任务模型）	0.21–0.99

回放依赖两条假设（给定难度后各回合独立；任务难度服从同一 Beta），且 DeepSeek 的 SkillOpt/GEPA 都从种子**最低的那次**开发得分起算；若起点随机抽取，τ^3 上的报告增益就变得少见（P = 0.02–0.17）。

单次赢就收：误收未改动候选	24–41%
要求赢两次：误收 / 真 +2 任务保留	6–17% / 55–94%

模拟结果，不是在新优化循环里实测。

小票 03 · 覆盖面 §5.2 Finding 3 · 附录 F.2

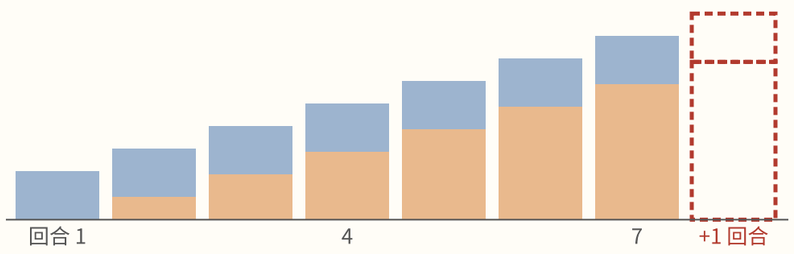
技能让会做的更稳，不让新的变会

种子 vs 无技能：新覆盖 / 丢失任务	36 / 13（p = .001）
学到的技能 vs 种子：新覆盖 / 丢失	57 / 62

“覆盖”= 三次测试至少成功一次。成功率上升的任务里，三次回合的工具序列更一致（Mantel–Haenszel OR 1.61，p = .03）。

小票 04 · 计费方式 §4.2–4.4 · Prop. 1 · Finding 8

多一回合，多付的不止一回合 示意



每回合都把前缀和全部历史重发一遍：回合多 1%，输入 token 至少多 1%，长回合接近 2%。实测同一任务内回合每多 10%，输入 token 多约 14–16%（斜率 1.35–1.58），而输出 token 与工具调用斜率 ⩽ 1.03。

DeepSeek 9 个技能多出的 log 输入 token：来自多出的回合	73%
来自更长的技能文本	21%
59 次编辑：回合变化 / 工具调用变化 / 文本长度变化解释的方差	93% / 79% / 10%

示意图为本项目按原文 Eq.(3) 绘制的阶梯，不是实验数据。四项拆分只在双方 6 个回合都可用的任务上算；按字符比例把 token 分摊到上下文组件。

小票 05 · 编辑器写了什么 §5.3 Findings 6–7 · 附录 E.1–E.3

只加规则，没有一条叫它少读

59 次可追溯父代的编辑中，让技能变长的	58
DeepSeek 9 个技能新增规则（每个平均 37 条，最多 105）	337
其中“再核对一遍”（verify）	52%
“一次只做一个调用”（serialize）	18 条
“少读 / 只读需要的部分”	0 条

被保留 vs 被拒的编辑，输入 token 平均涨幅	~10% vs ~14%（p = .46）
不降开发成功率又不增 token 的编辑	5 / 59
AutoBench：写操作与别的写操作同回合（种子 → 学到的技能）	56% → ⩽ 0.3%

串行写有其道理：AutoBench 每次写都整体读改存模拟世界，并发写会互相覆盖——把所有调用按种子的密度打包，AutoBench 的 log ρ 从 0.17 减半到 0.08，大头正是这些写。也就是说，并非每个多出的回合都是浪费。

换模型、换基准：增益不跟走，费用跟走

迁移的 9 个技能只有 1 个高于种子（+1.9）；DeepSeek 学的 τ^3 技能在 Luna 上仍读 1.08–1.38×，TB2 技能在 AutoBench 上读 1.24–1.49×。

作者建议 · 四条设计原则 §6

1. 增益超过“仅靠挑选会报出的增益”才收；分不出成功率时，按超出运行间差异的 token 差来定。
2. 按编辑**新增的回合**计费，而不是按文本长度。
3. 允许编辑器**删除与合并**规则（DeepSeek 59 次编辑只缩短过 1 次）。
4. 让编辑器看到之前规则**花了多少回合与 token**。

账单换算也不翻转结论：按缓存价计费仍为种子的 1.05–1.27×；按“只付本回合新增历史”重算（压缩历史的近似），回合斜率降到约 1.04，但每个技能仍是 1.05–1.36×。

附加实验 · 三词种子 附录 C.5（Luna，各一次）

从“Solve the task.”出发会怎样

AutoBench：三词 → O / G	25.0 → 43.3 / 44.4
对照：从手写种子出发	42.8 / 41.4
τ^3：返回技能	60.0 / 68.0
对照：三词种子两次测试	65.3 / 68.0

一次改写就追平从手写种子出发的结果，再改写也不再提高；AutoBench 上 18.3 / 19.4 点增益中各有 16.7 点来自无技能或手写种子至少做对过一次的 63 个任务。τ^3 上的开发增益（+10.0 / +6.7）回放达到概率 0.27 / 0.74。

边界 · 本项目阅读 Limitations · 附录 A、C

- 每个“优化器×模型×基准”只跑了**一次**优化；三次测试只反映执行噪声，不反映优化随机性。
- 一个 harness（OpenCode，不压缩历史）、两个模型、同一个种子技能；规则 → 行为是对应关系，没有做单条规则的受控改写。
- Luna 的 AutoBench 增益是真的；作者的“分不出成功率就看成本”规则在那里会退回种子，也会错过它。

本项目读法：“开发增益落在挑选噪声里”与第三期的 2610.28798、2610.30499 是同一判断，本篇的独立价值在后半——成本藏在**回合**里而不是技能长度里，且成功率指标看不见它。读法来自卡片 diagnose_self_improvement / evidence_gated_retention，非作者原话。